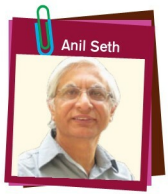

Spark: Exploring Big Data on a Desktop

Here's an introduction to Apache Spark, a very fast tool for large scale data processing. Spark is easy to use, and runs on Hadoop and Mesos as a standalone application or on the cloud. Applications can be quickly written in Java, Scala or Python. Developers boast that it can run programs 100 times faster than MapReduce in memory, or 10 times faster on disk.

My interest in Hadoop had been sparked a year ago by the following headline: "Up to 100x faster than Hadoop MapReduce." Now, it's finally time to explore Apache Spark.

MapReduce expects you to write two programs, a mapper and a reducer. The MapReduce system will try to run the mapper programs on nodes close to the data. The output of various mapper programs will be key value pairs. The MapReduce system will forward the output to various reducer programs, based on the key.

In the Spark environment, you write only one set of code containing both the mapper and reducer code. The framework will distribute and execute the code in a manner that will try to optimise the performance and minimise the movement of data over the network. Besides, the program could include additional mappers and reducers to process the intermediate results.

Installing Spark

You will need to download the code from the Apache Spark site (*http://spark.apache.org/*) based on the version of Hadoop on your system. Installation involves just unzipping the downloaded file. The documentation will guide you on how to configure it for a cluster, e.g., consisting of three nodes – *h-mstr, h-slv1* and *h-slv2*.

Getting a taste of Spark with word count

Let's suppose that you have already created and copied text files in the Hadoop Distributed File System (HDFS) in the directory */user/fedora/docs/*.

Start the Python Spark shell, as follows:

```
$ bin/pyspark
```

Open the text files in HDFS and run the usual word count example:

```
>>> data = sc.textFile('hdfs://h_mstr/user/fedora/docs')
>>> result = data.flatMap(lambda line:re.sub('[^a-z0-9]+',' ',line.lower()).split())
                        .map(lambda x:(x,1))
                        .reduceByKey(lambda a,b:a+b)
>>> output=result.collect()
>>> output.sort()
>>> for w,c in output:
>>>     print('%s %d'%(w,c))
```

The shell creates a Spark context, *sc*. Use it to open the files contained in the HDFS *docs* directory for the user Fedora.

You use *flatMap* to split each line into words. In this case, you have converted each line into lower case and replaced all non-alphanumeric characters by spaces, before splitting it into words. Next, map each word into a pair (word, 1).

The mapping is now complete and you can reduce it by keyword, accumulating the count of each word. So far, the response would have been very fast. Spark is lazy and evaluates only when needed, which will be done when you run the *collect* function. It will return a list of word count pairs, which you may sort and print.

Finding out which files contain a particular word

Let's assume that you have a large number of small text files and you would like to know how frequently a particular word occurs in various files.

Spark has the option to process the whole file rather than one record at a time. Each file is treated as a pair of values – the file name and the content.

```
from pyspark import SparkContext
import re
# input to wholeFile is a file name, file content pair
def wholeFile(x):
    name=x[0]
    words  = re.sub('[^a-z0-9]+',' ',x[1].lower()).split()
    return [(word,name) for word in set(words)]
```